



**Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)**

**Факультет «Информатика и системы управления»  
Кафедра «Системы обработки информации и управления»**

**Рубежный контроль №1  
по дисциплине «Парадигмы и конструкции языков программирования»**

Выполнил:  
студент группы ИУ5-33Б Павлинич А. А.  
подпись \_\_\_\_\_  
дата: 03.11.2025

Проверил:  
Гапанюк Ю. Е.  
подпись \_\_\_\_\_  
дата:

2025 г.

Листинг кода:

```
from typing import List
```

```
class SchoolClass:
```

```
    def __init__(self, id: int, name: str):
        self.id = id
        self.name = name
```

```
    def __repr__(self):
        return f"SchoolClass(id={self.id}, name='{self.name}')
```

```
class Student:
```

```
    def __init__(self, id: int, last_name: str, grade_point_average: float, class_id: int):
        self.id = id
        self.last_name = last_name
        self.grade_point_average = grade_point_average
        self.class_id = class_id
```

```
    def __repr__(self):
        return f"Student(id={self.id}, last_name='{self.last_name}', gpa={self.grade_point_average},
class_id={self.class_id})"
```

```
class StudentClassRelation:
```

```
    def __init__(self, student_id: int, class_id: int):
        self.student_id = student_id
        self.class_id = class_id
```

```
    def __repr__(self):
        return f"StudentClassRelation(student_id={self.student_id}, class_id={self.class_id})"
```

```
class DataProcessor:
```

```
    def __init__(self, classes: List[SchoolClass], students: List[Student], student_relations:
List[StudentClassRelation]):
        self.classes = classes
        self.students = students
        self.student_relations = student_relations
```

```
    def task1_one_to_many(self) -> List[tuple]:
        """Список всех школьников, у которых фамилия начинается с буквы «А», и названия их
классов"""
        result = []
        for student in self.students:
            if student.last_name.startswith('A'):
                school_class = next((cls for cls in self.classes if cls.id == student.class_id), None)
                if school_class:
                    result.append((student.last_name, school_class.name))
        return result
```

```
    def task1_one_to_many_comprehension(self) -> List[tuple]:
```

```

        """С использованием list comprehension"""
        return [
            (student.last_name, school_class.name)
            for student in self.students
            for school_class in self.classes
            if student.class_id == school_class.id and student.last_name.startswith('A')
        ]

def task2_min_gpa_by_class(self) -> List[tuple]:
    """Список классов с минимальным средним баллом школьников в каждом классе,
    отсортированный по минимальному баллу"""
    class_students = {}
    for student in self.students:
        if student.class_id not in class_students:
            class_students[student.class_id] = []
        class_students[student.class_id].append(student.grade_point_average)

    result = []
    for class_id, gpas in class_students.items():
        school_class = next((cls for cls in self.classes if cls.id == class_id), None)
        if school_class:
            min_gpa = min(gpas)
            result.append((school_class.name, min_gpa))

    # Сортируем по минимальному баллу
    return sorted(result, key=lambda x: x[1])

def task2_min_gpa_comprehension(self) -> List[tuple]:
    """С использованием comprehensions и функций высшего порядка"""
    class_gpas = {}
    for student in self.students:
        class_gpas.setdefault(student.class_id, []).append(student.grade_point_average)

    return sorted([
        (next(cls for cls in self.classes if cls.id == class_id).name, min(gpas))
        for class_id, gpas in class_gpas.items()
    ], key=lambda x: x[1])

def task3_many_to_many(self) -> List[tuple]:
    """Список всех связанных школьников и классов, отсортированный по школьникам"""
    connections = []
    for rel in self.student_relations:
        student = next((s for s in self.students if s.id == rel.student_id), None)
        school_class = next((cls for cls in self.classes if cls.id == rel.class_id), None)
        if student and school_class:
            connections.append((student.last_name, school_class.name))

    return sorted(connections, key=lambda x: x[0])

def task3_many_to_many_comprehension(self) -> List[tuple]:

```

```

        """С использованием list comprehension"""
        return sorted([
            (student.last_name, school_class.name)
            for rel in self.student_relations
            for student in self.students if student.id == rel.student_id
            for school_class in self.classes if school_class.id == rel.class_id
        ], key=lambda x: x[0])

def main():

    classes = [
        SchoolClass(1, "5А"),
        SchoolClass(2, "6Б"),
        SchoolClass(3, "7В"),
        SchoolClass(4, "8А"),
        SchoolClass(5, "9Б")
    ]

    students = [
        Student(1, "Андреев", 4.5, 1),
        Student(2, "Петров", 3.8, 2),
        Student(3, "Александрова", 4.7, 1),
        Student(4, "Сидоров", 3.2, 3),
        Student(5, "Антонов", 4.9, 4),
        Student(6, "Кузнецова", 4.1, 2),
        Student(7, "Абрамова", 4.3, 5),
        Student(8, "Иванов", 3.9, 3),
        Student(9, "Алексеев", 4.6, 4),
        Student(10, "Морозов", 3.5, 5)
    ]

    student_relations = [
        StudentClassRelation(1, 1), # Андреев - 5А
        StudentClassRelation(1, 3), # Андреев - 7В (дополнительные занятия)
        StudentClassRelation(2, 2), # Петров - 6Б
        StudentClassRelation(3, 1), # Александрова - 5А
        StudentClassRelation(3, 4), # Александрова - 8А (факультатив)
        StudentClassRelation(4, 3), # Сидоров - 7В
        StudentClassRelation(5, 4), # Антонов - 8А
        StudentClassRelation(6, 2), # Кузнецова - 6Б
        StudentClassRelation(7, 5), # Абрамова - 9Б
        StudentClassRelation(8, 3), # Иванов - 7В
        StudentClassRelation(8, 1), # Иванов - 5А (репетиторство)
        StudentClassRelation(9, 4), # Алексеев - 8А
        StudentClassRelation(9, 5), # Алексеев - 9Б (подготовка к экзаменам)
        StudentClassRelation(10, 5), # Морозов - 9Б
    ]

    processor = DataProcessor(classes, students, student_relations)

```

```

print("=" * 70)
print("ЗАДАНИЕ 1: Школьники с фамилией на 'А' и их классы")
print("=" * 70)
result1 = processor.task1_one_to_many()
for last_name, class_name in result1:
    print(f"{last_name} - {class_name}")

print("\n" + "=" * 70)
print("ЗАДАНИЕ 2: Классы с минимальным средним баллом (сортировка по баллу)")
print("=" * 70)
result2 = processor.task2_min_gpa_by_class()
for class_name, min_gpa in result2:
    print(f"{class_name}: {min_gpa} баллов")

print("\n" + "=" * 70)
print("ЗАДАНИЕ 3: Все связи школьник-класс (сортировка по школьникам)")
print("=" * 70)
result3 = processor.task3_many_to_many()
for last_name, class_name in result3:
    print(f"{last_name} - {class_name}")

print("\n" + "=" * 70)
print("Реализация с использованием comprehensions")
print("=" * 70)

print("\nЗадание 1 (list comprehension):")
result1_alt = processor.task1_one_to_many_comprehension()
for last_name, class_name in result1_alt:
    print(f"{last_name} - {class_name}")

print("\nЗадание 2 (comprehension + min):")
result2_alt = processor.task2_min_gpa_comprehension()
for class_name, min_gpa in result2_alt:
    print(f"{class_name}: {min_gpa} баллов")

print("\nЗадание 3 (list comprehension):")
result3_alt = processor.task3_many_to_many_comprehension()
for last_name, class_name in result3_alt:
    print(f"{last_name} - {class_name}")

if __name__ == "__main__":
    main()

```

Результат выполнения программы:

```
=====
ЗАДАНИЕ 1: Школьники с фамилией на 'А' и их классы
=====
Андреев - 5А
Александрова - 5А
Антонов - 8А
Абрамова - 9Б
Алексеев - 8А

=====
ЗАДАНИЕ 2: Классы с минимальным средним баллом (сортировка по баллу)
=====
7В: 3.2 баллов
9Б: 3.5 баллов
6Б: 3.8 баллов
5А: 4.5 баллов
8А: 4.6 баллов

=====
ЗАДАНИЕ 3: Все связи школьник-класс (сортировка по школьникам)
=====
Абрамова - 9Б
Александрова - 5А
Александрова - 8А
Алексеев - 8А
Алексеев - 9Б
Андреев - 5А
Андреев - 7В
Антонов - 8А
Иванов - 7В
Иванов - 5А
Кузнецова - 6Б
Морозов - 9Б
Петров - 6Б
Сидоров - 7В
```

```
=====
Реализация с использованием comprehensions
=====

Задание 1 (list comprehension):
Андреев - 5А
Александрова - 5А
Антонов - 8А
Абрамова - 9Б
Алексеев - 8А

Задание 2 (comprehension + min):
7В: 3.2 баллов
9Б: 3.5 баллов
6Б: 3.8 баллов
5А: 4.5 баллов
8А: 4.6 баллов

Задание 3 (list comprehension):
Абрамова - 9Б
Александрова - 5А
Александрова - 8А
Алексеев - 8А
Алексеев - 9Б
Андреев - 5А
Андреев - 7В
Антонов - 8А
Иванов - 7В
Иванов - 5А
Кузнецова - 6Б
Морозов - 9Б
Петров - 6Б
Сидоров - 7В
```